Linux System Diagnostics Toolkit



Team Members:

2510080029-Sudeeptha

2510080037-Lokrey Latika

2510080043-Sai Siri

2510080061-Kovidha

Under the guidance of Dr.Kalpana Mam

Department of AI&DS

KLH University

PROJECT OVERVIEW

What is our project?

- Command-line Linux diagnostic toolkit
- Developed using C
- Runs on Ubuntu/WSL
- Provides system information and command execution
- Demonstrates Operating System concepts



PROJECT OBJECTIVES

01 Monitor Linux system information

02 Display uptime and memory

03 Execute Linux commands

04 Implement process management

05 Implement signal handling

06 Implement IPC using pipes

07 Implement file redirection

08 Implement multithreading

TECHNOLOGY STACK

OS

Ubuntu / WSL

LANGUAGE

C

COMPILER

GCC

BUILD

Makefile

DEBUGGING

GDB, Valgrind

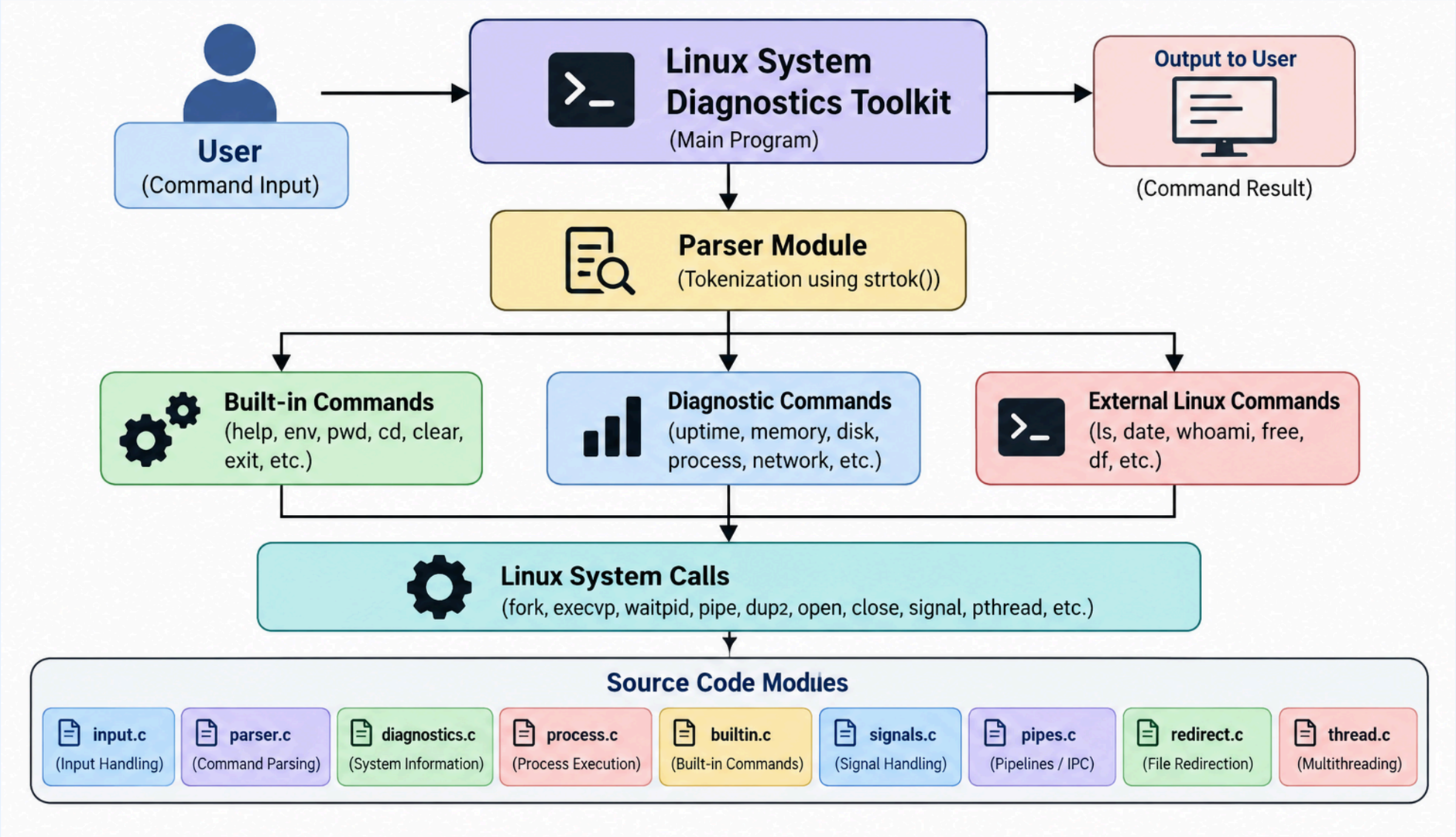
VERSION CONTROL

Git & GitHub

OS CONCEPTS

fork, exec, pipe, dup2, pthread

PROJECT ARCHITECTURE



WEEKS 1–5 DEVELOPMENT SUMMARY

Week 1:

- Ubuntu setup
- Project structure
- Uptime and RAM monitoring

Week 2:

- Dynamic memory management
- Modular programming

Week 3:

- Command parser using strtok()

Week 4:

- Process creation using fork()
- Command execution using execvp()

Week 5:

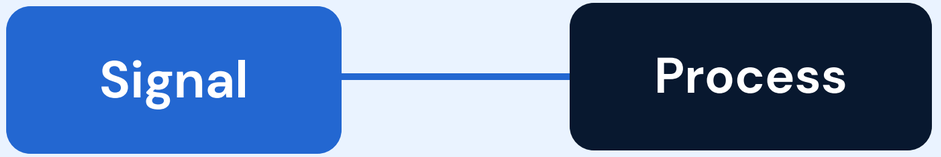
- Built-in commands
- Environment variables

WEEK 6: SIGNALS & PROCESS CONTROL

IMPLEMENTED

- signal()
- SIGINT
- SIGCHLD
- Ctrl+C handling
- Child process management

```
diagnostics>
^C
Signal handled safely
```



WEEK 7: PIPES & IPC

IMPLEMENTED

pipe()

fork()

dup2()

Inter-process communication



TESTED COMMANDS

```
ls | wc
free -h | grep Mem
ps -ef | grep bash
```

WEEK 8: DEBUGGING & MEMORY MANAGEMENT

GDB FEATURES

Breakpoints

step

next

backtrace

VALGRIND FEATURES

Memory leak detection

Invalid memory access detection

```
$ valgrind ./diagnostics
definitely lost: 0 bytes
ERROR SUMMARY: 0 errors

(gdb) break main
(gdb) run
Breakpoint 1, main ()
```

WEEK 9: FILE REDIRECTION

IMPLEMENTED

File descriptors

open()

close()

dup2()



EXAMPLES

```
ls > files.txt
free -h > memory.txt
echo Test >> log.txt
```

WEEK 10: POSIX THREADS

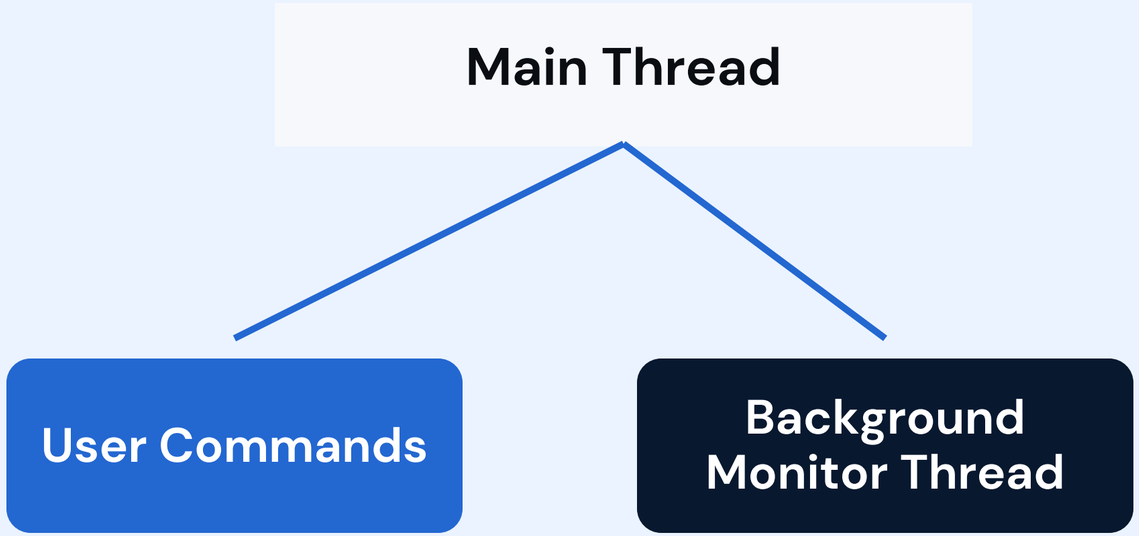
IMPLEMENTED

pthread_create()

pthread_detach()

Background monitoring thread

Concurrent execution




```
[Monitor] Linux Diagnostics Toolkit
Running...
```

FINAL TESTING & DEMONSTRATION

```
$ uptime
$ memory
$ help
$ pwd
$ env
$ ls
$ ls | wc
$ free -h | grep Mem
$ ls > files.txt
```

✓ Compilation successful

✓ Pipes working

✓ Redirection working

✓ Signals working

✓ Threads working

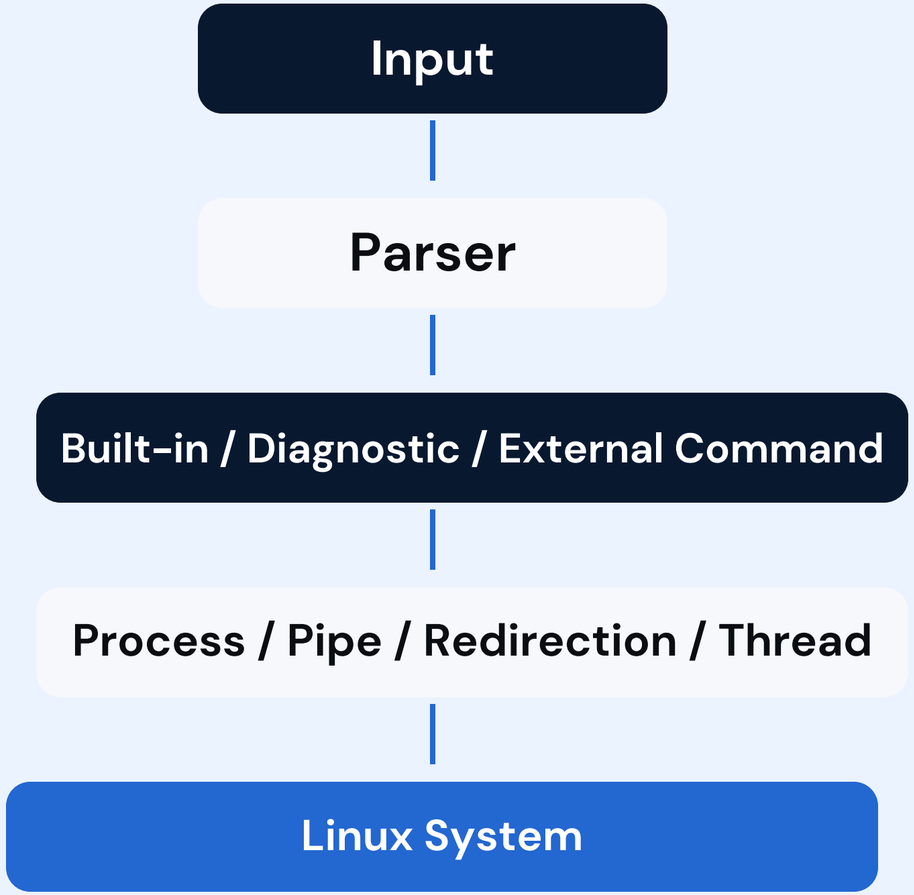
✓ Valgrind checked

✓ GDB tested

PROJECT OUTCOME

WHAT WE ACHIEVED

- Modular Linux diagnostic toolkit
- Process creation and management
- Signal handling
- Inter-process communication
- File I/O redirection
- Multithreading
- Debugging and memory analysis



FUTURE ENHANCEMENTS

- CPU Usage Monitoring
- Disk Usage Analysis
- Network Diagnostics
- Process Monitoring
- System Log Analysis
- Real-Time Dashboard
- Automated Health Alerts
- Diagnostic Report Generation

THANK YOU